

Веб-приложение для записи на проекты

Результаты второй итерации

План итерации 2:

- **Бэкенд**

- Реализовать JWT-авторизацию (модуль Auth): эндпоинты /auth/login, /auth/register, защита маршрутов через Guards.
- Подключить Prisma ORM к существующей MySQL, создать модели на основе ER-диаграммы.
- Разработать CRUD для курсов: GET /courses, POST /courses, GET /courses/{id}, PUT /courses/{id} с валидацией и проверкой прав.
- Написать юнит-тесты для сервисов курсов.

- **Фронтенд**

- Реализовать формы логина и регистрации с валидацией, сохранять токен в Pinia store.
- Создать страницу управления курсами (только для преподавателя): список, создание, редактирование.
- Интегрировать страницы с реальным API (замена моков).
- Добавить уведомления об успехе/ошибке (через Naive UI).

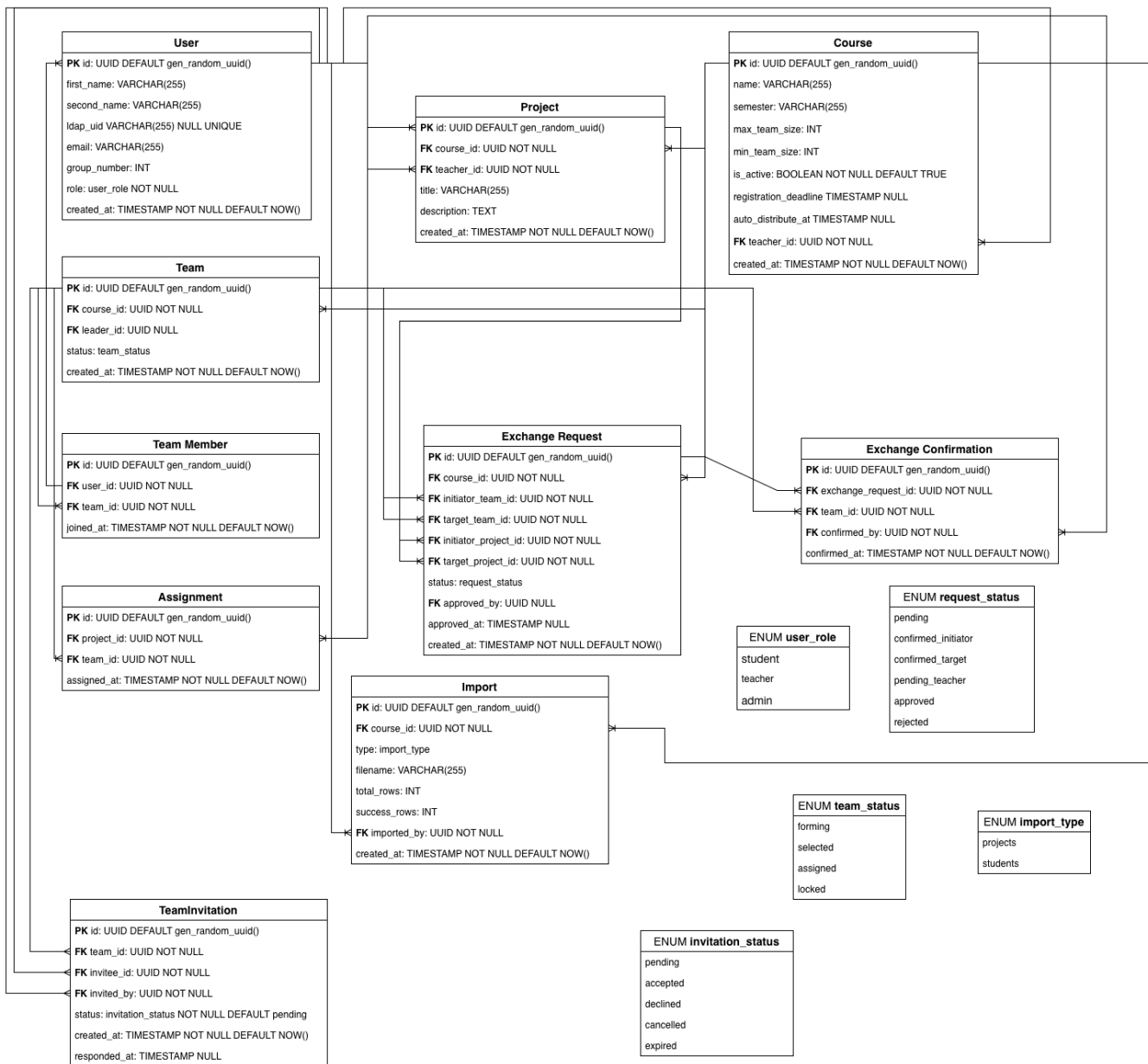
План итерации 2:

- **База данных**
 - Написать миграции для всех таблиц и наполнить тестовыми данными.
 - Создать seed-скрипт для первоначального наполнения (преподаватель, несколько курсов, проектов).
- **DevOps**
 - Доработать Docker-окружение: добавить сценарий для разработки с hot-reload (тома для кода).
 - Настроить GitHub Actions для автоматической проверки линтера и сборки при пушах в main.
- **Документация**
 - Обновить OpenAPI спецификацию с учётом новых эндпоинтов.
 - Дополнить README разделом «Запуск с Docker» и инструкцией по настройке переменных окружения.

Результаты итерации 2

- **Бэкенд:**
 - JWT-авторизация: создан модуль Auth, эндпоинты /auth/login, @SkipAuth, JwtAuthGuard, RolesGuard, декоратор @Roles.
 - Подключён Prisma ORM: схемы описаны, PrismaService интегрирован, клиент генерируется в backend/src/generated.
 - CRUD для курсов: реализованы все эндпоинты с валидацией (DTO) и проверкой прав (только admin может создавать/обновлять/удалять). Заменены моки на работу с БД.
 - Юнит-тесты для сервисов курсов - подготовлена база, но не все тесты дописаны (запланировано на следующую итерацию).
 - Добавлен модуль Health (/health, /health/ready) для мониторинга.
- **Фронтенд:**
 - Формы логина и регистрации, валидация, Pinia store для хранения токена и роли.
 - Страница управления курсами: CoursesListPage.vue со списком, кнопка “Добавить курс” ведёт на отдельную страницу создания; редактирование – на странице деталей; удаление находится в деталях курса.
 - Интеграция с API: вызовы coursesApi, authApi работают, токен подставляется.
 - Уведомления: глобальные уведомления через useNotification Naive UI (успех/ошибка).
 - Защита маршрутов: гард на requiresAdmin для страниц создания/редактирования курса.

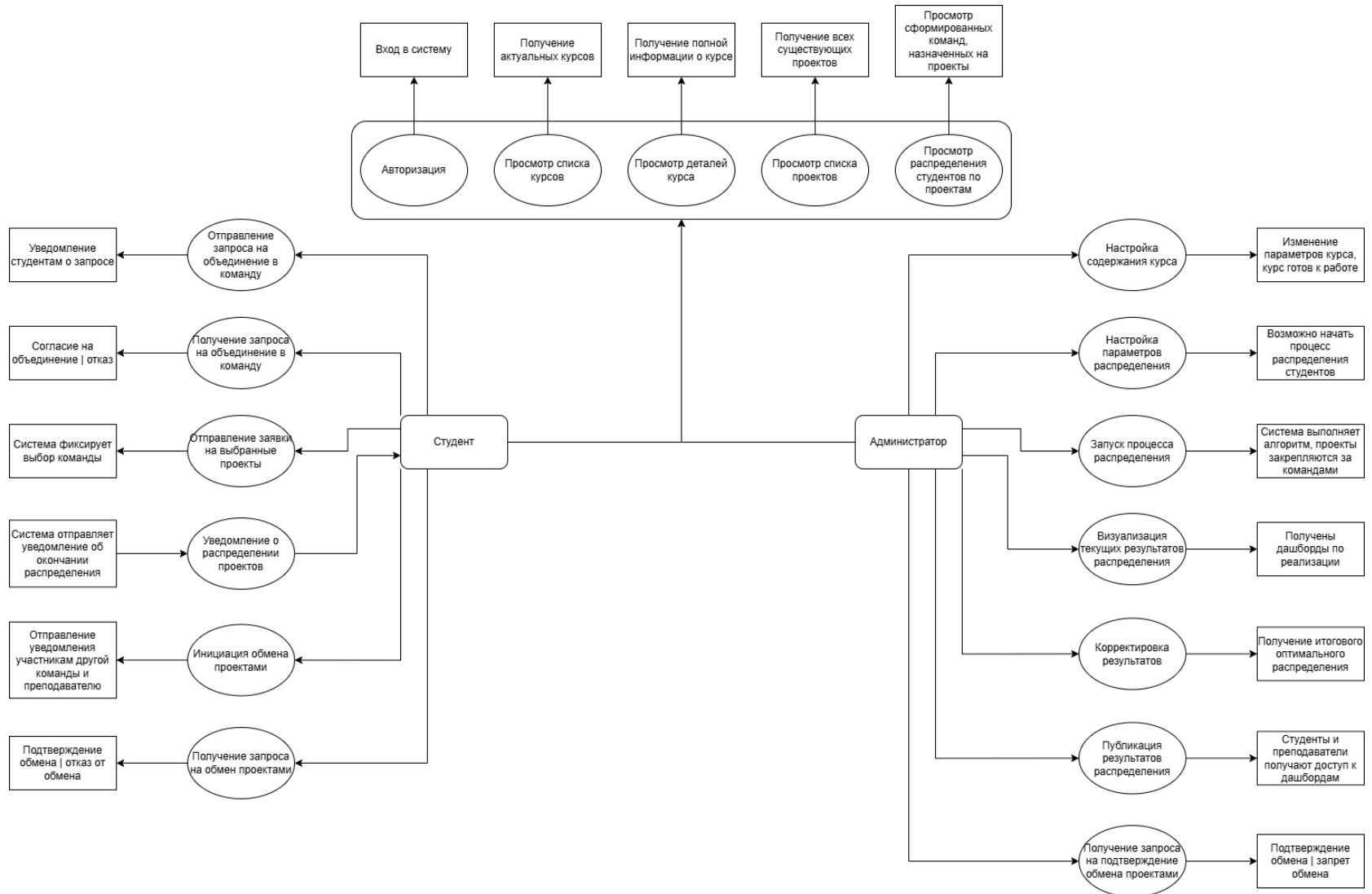
ER-диаграмма



Результаты итерации 2

- **База данных**
 - Миграции для всех таблиц: созданы миграции для пользователей, курсов, проектов, команд, приглашений, назначений, обменов, импортов.
 - Seed-скрипт: написан (prisma/seed.ts), добавляет тестового администратора и студентов, курсы, проекты.
- **DevOps**
 - Docker для разработки: добавлен docker-compose.dev.yaml, Dockerfile.dev для backend и frontend с монтированием томов и hot-reload.
 - GitHub Actions: настроен CI (ci.yml) с запуском линтера, проверки типов и сборки для всех пакетов.
- **Документация**
 - OpenAPI спецификация обновлена: добавлены эндпоинты авторизации, курсов (CRUD), проектов (частично).
 - README дополнен: подробные инструкции по запуску с Docker (production и development), описаны все переменные окружения.

Use-case диаграмма



План на итерацию 3

- **Бэкенд**
 - CRUD для проектов (модуль ProjectsModule).
 - Управление командами и участниками (модули TeamsModule, TeamMembersModule).
 - Выбор проекта командой (эндпоинт POST /teams/{teamId}/choose-project).
 - Импорт данных из CSV (POST /courses/{courseId}/import).
 - Интеграционные тесты для всех ключевых эндпоинтов.
- **Фронтенд**
 - Страница управления проектами (преподаватель).
 - Страница команд и выбор проекта (студент).
 - Страница импорта данных (преподаватель).

План на итерацию 3

- **База данных**
 - Доработка схемы (при необходимости).
 - Улучшение seed-скрипта (больше данных, покрытие всех сценариев).
- **DevOps**
 - Добавить в CI запуск интеграционных тестов.
 - Оптимизировать Docker-сборку.
 - Сделать проксирование через nginx вместо контейнера.
- **Документация**
 - Обновить OpenAPI (проекты, команды, импорт).